INTELLIGENT

TOHOKU
UNIVERSITY

GROUP MEMBERS

<TESP_2025>

- Charlotte L. Primiceri
- Davide de Ciutiis
- Serena Trovalusci
- Harini Satyavada
- Vishnu Anand

INTRODUCTION:

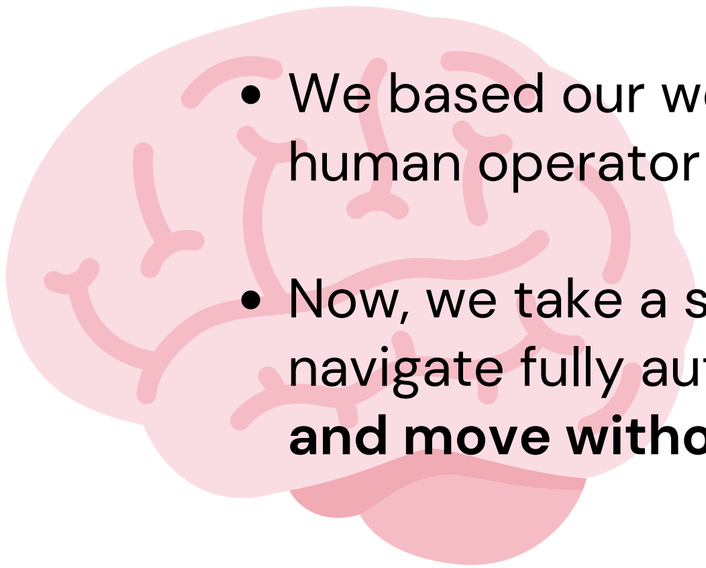
From Gesture-Controlled to Autonomous Snake Locomotion

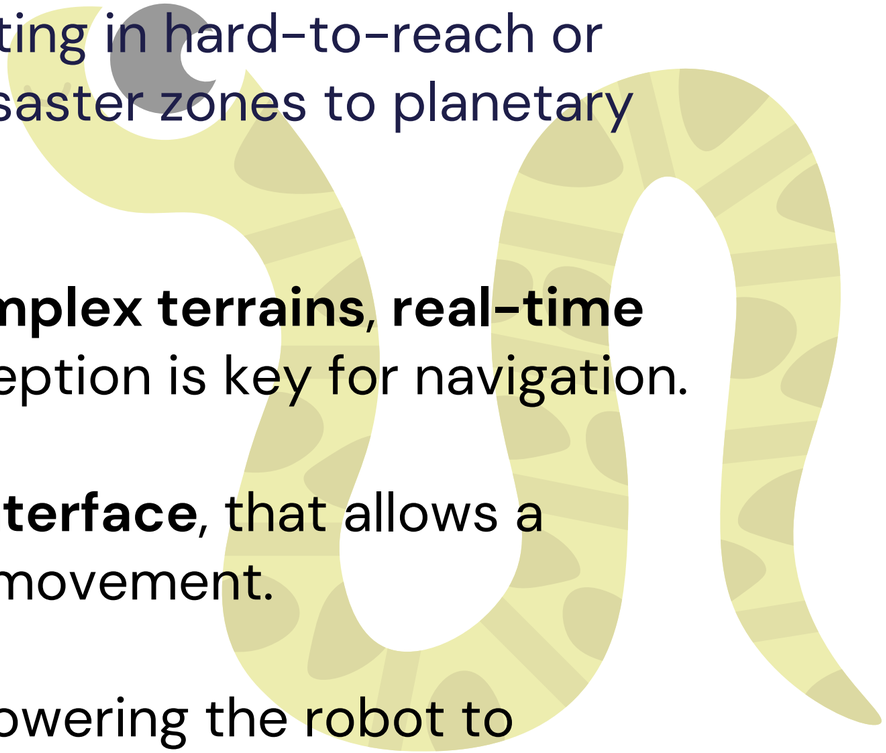
- **Snake robots** are ideal for exploring and operating in hard-to-reach or dangerous or hazardous environments, from disaster zones to planetary surfaces.

- When the snake robot is used on **unknown complex terrains**, **real-time gait adjustment** based on environmental perception is key for navigation.

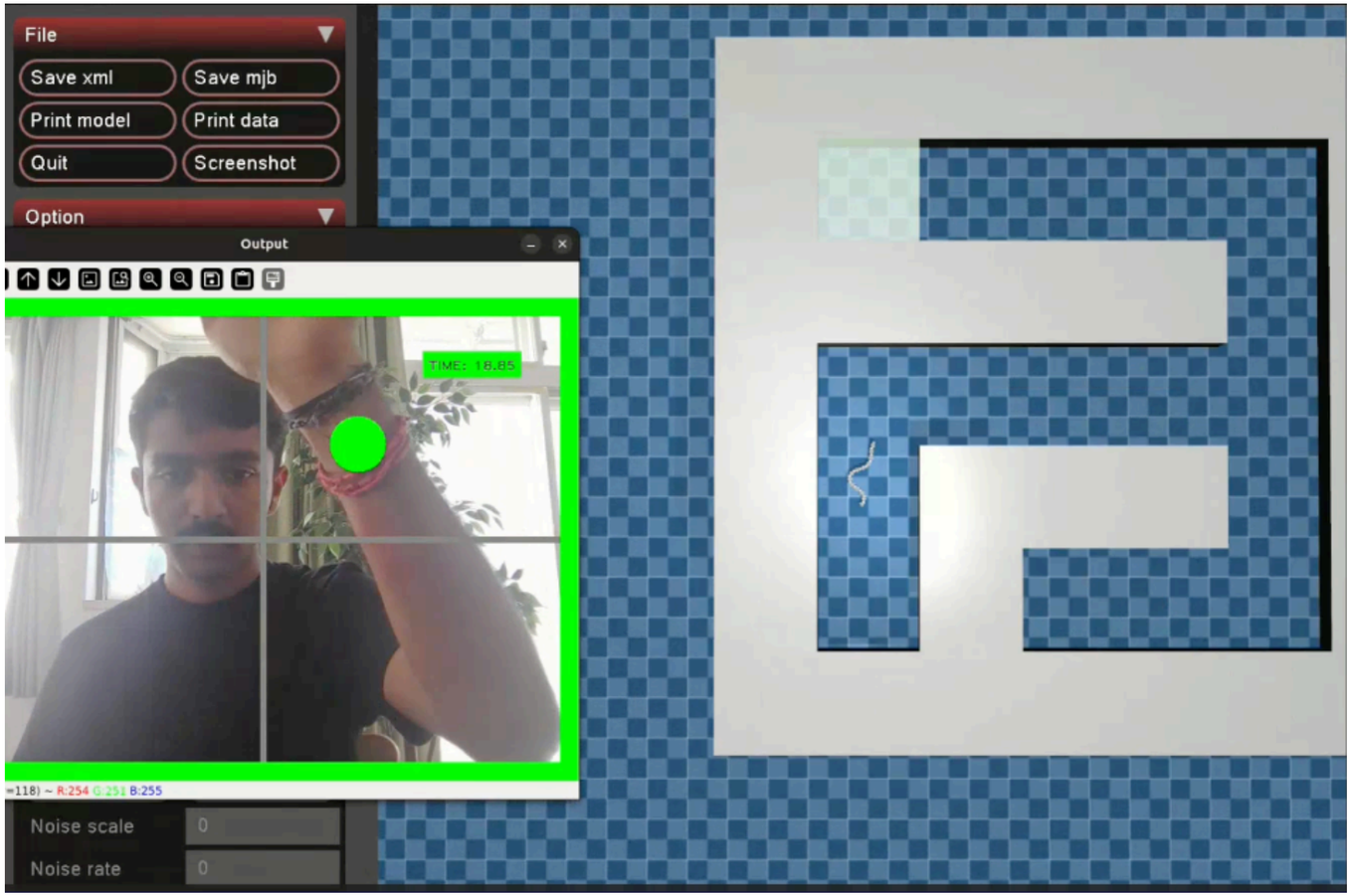
- We based our work on a **gesture-controlled interface**, that allows a human operator to manually guide the robot's movement.

- Now, we take a significant step forward — empowering the robot to navigate fully autonomously using artificial intelligence to **perceive, plan, and move without human intervention.**



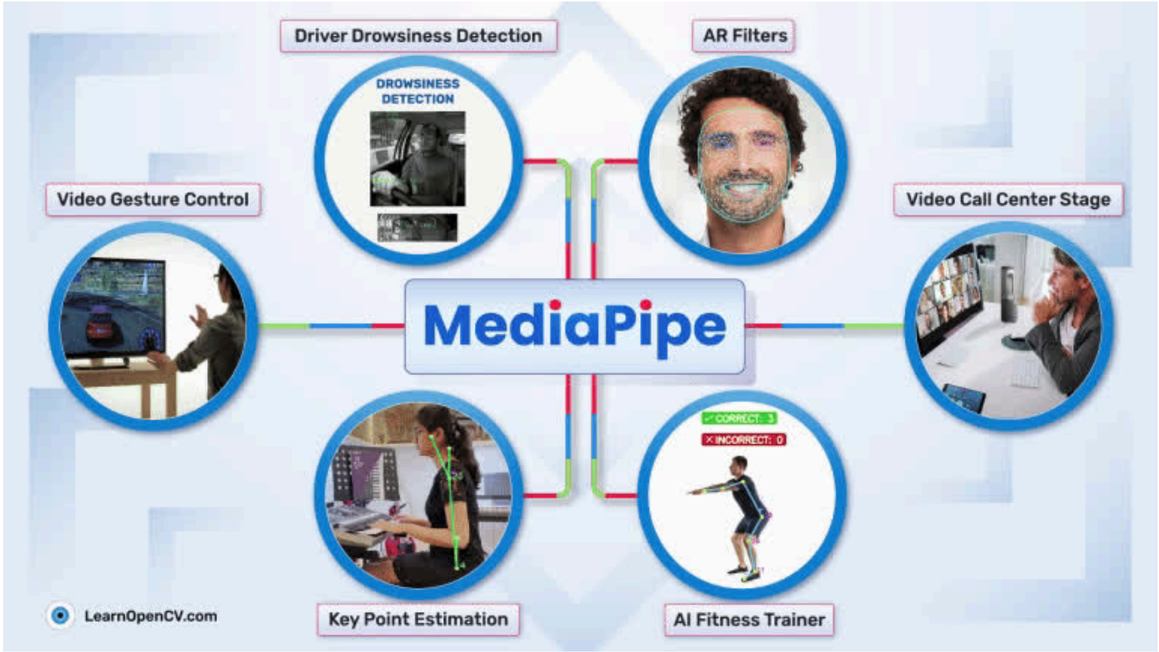


GESTURE-CONTROLED SNAKE



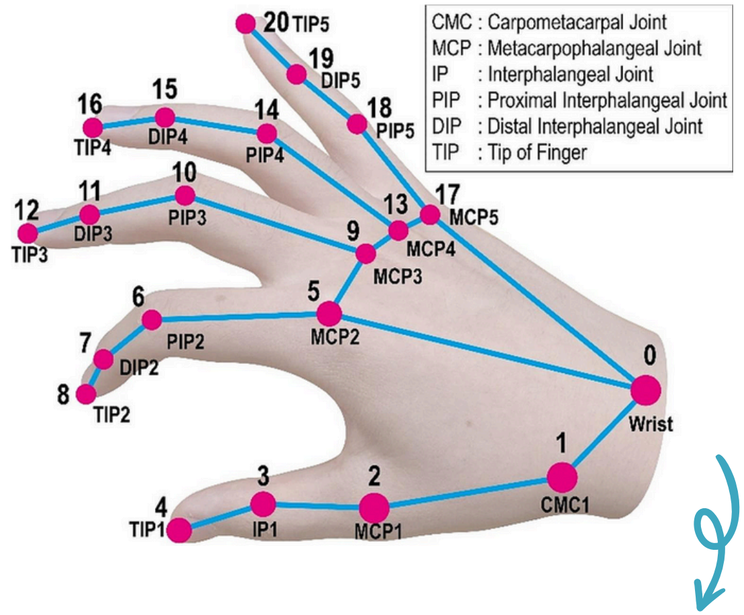
DETAILS: **MEDIAPIPE**

MediaPipe is an open-source framework from Google for building pipelines to perform computer vision inference over arbitrary sensory data such as video or audio



MEDIAPIPE: **POSE LANDMARK DETECTION**

- The **MediaPipe Pose Landmarker** task lets you detect landmarks of human bodies in an image or video.
- This task uses machine learning (ML) models that work with single images or video.
- The task outputs body pose landmarks in image coordinates and in 3-dimensional world coordinates.




```
wanted_pose_landmarks = [
    PoseLandmark.RIGHT_WRIST,
    ]
```

MEDIAPIPE: **WORKFLOW**

WEBCAM INPUT

↓

WRIST LANDMARKS DETECTION AND POSITION EXTRACTION

↓

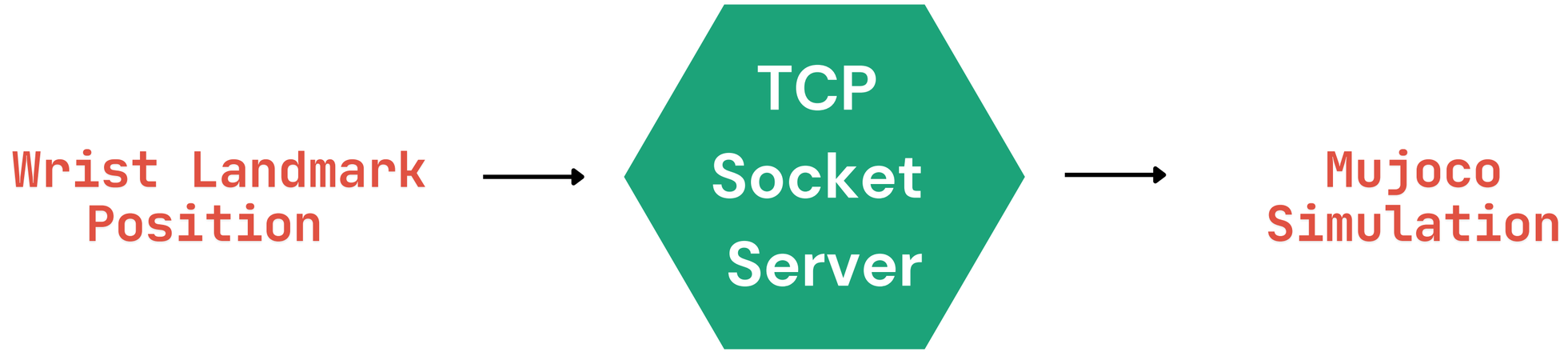
CHECKS (FRAME, HAND IN FRAME)

↓

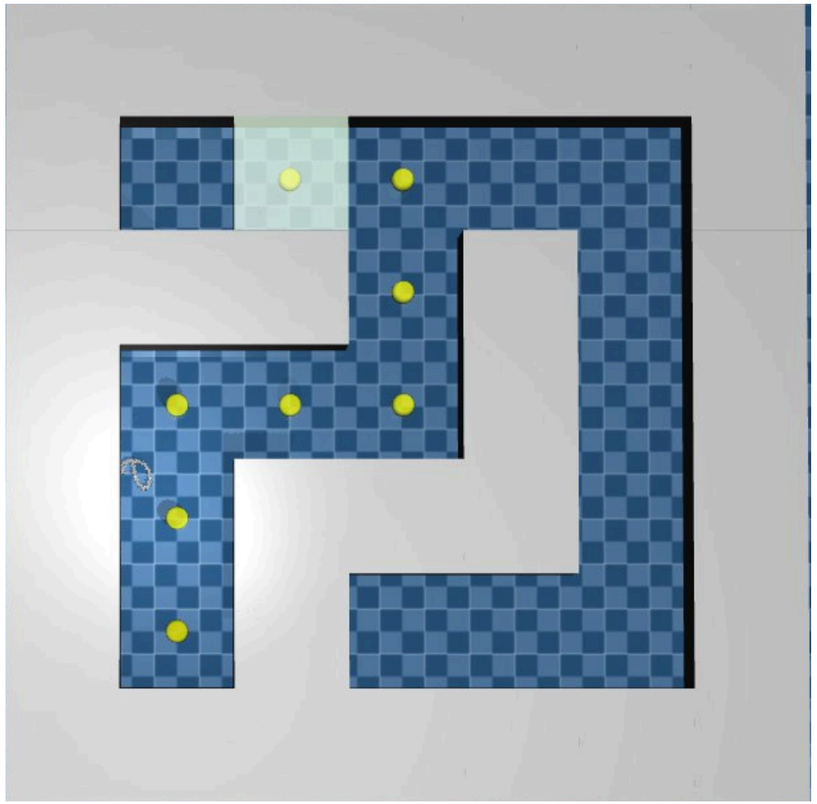
GESTURE SIGNAL → [x, y, is_hand_in_frame]

Real-Time Communication via Socket Server

A TCP socket server is used to enable real-time data exchange between the **MediaPipe gesture detection** client and the **Python MuJoCo control system**.



DETAILS: **MUJOCO**



```
model = mujoco.MjModel.from_xml_path(f"{script_dir}/src/scene_maze.xml")
data = mujoco.MjData(model)
```

MUJOCO: **GESTURE-TO-MOTION MAPPING**

INPUT

Gesture signal from server → **[x, y, is_hand_in_frame]**

PARAMETERS

$\omega = f(\mathbf{y})$ angular velocity
$B = f(\mathbf{x})$ offset
$\phi = \pi/4$ phase
$\theta_{new} = \theta_{old} + \omega t$ angle

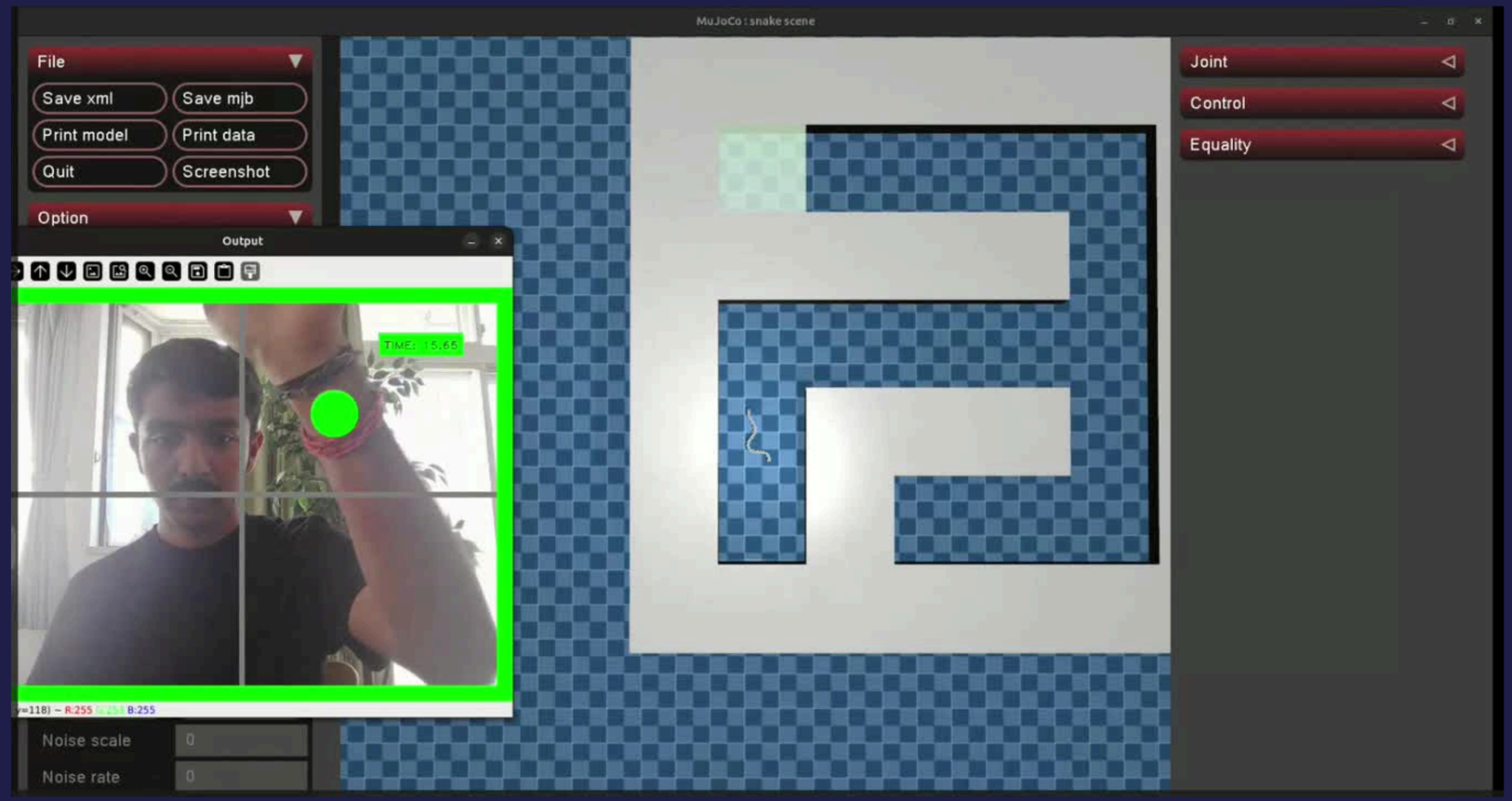
TARGET CONFIGURATIONS

target q[i]= amp*sin($\theta + \phi * i$) + B

JOINT CONTROL

data.ctrl[:] = target q (x, y)

RESULT: **GESTURE-CONTROL**

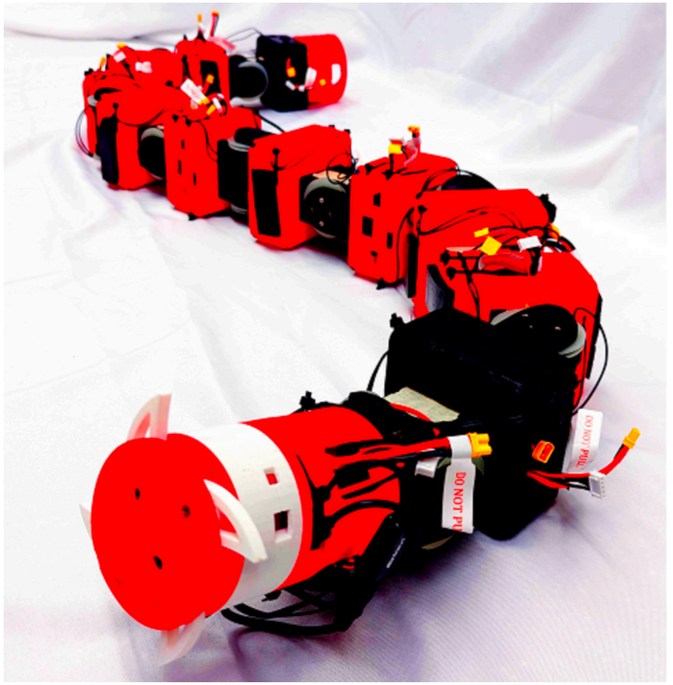


RESULT: **GESTURE-CONTROL**

TOWARDS AN INTELLIGENT SNAKE: **HIERARCHICAL CONTROL SCHEME**

- In order to obtain an adaptive and scalable navigation of a snake robot in complex and unstructured environments, we have used a **four-layer hierarchical control scheme** to enable the snake robot to navigate freely in large-scale environments. **[1]**

- Hierarchical control scheme: **global planning**, **local planning**, **gait generation**, and **gait tracking**.

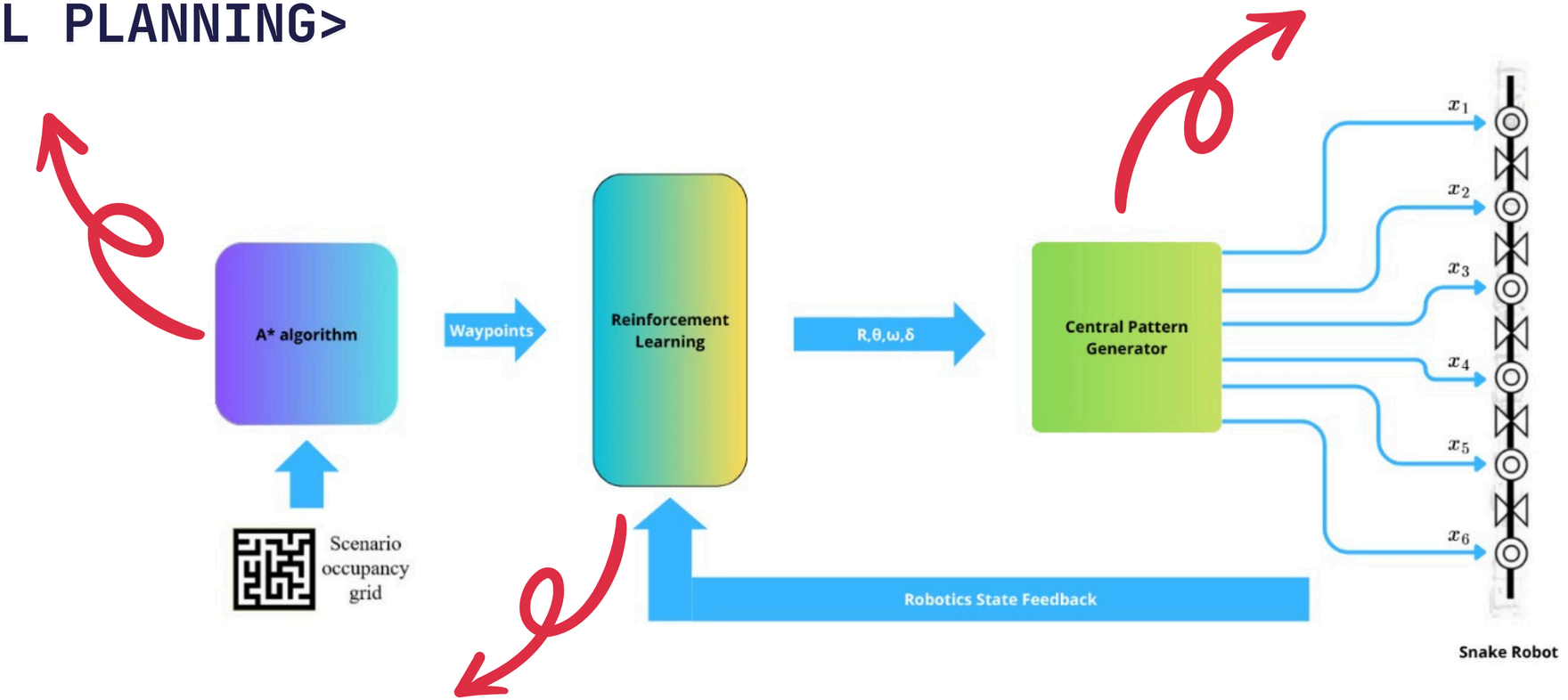


[1] "*Hierarchical RL-Guided Large-scale Navigation of a Snake Robot*" Shuo Jiang, Adarsh Salagame, Alireza Ramezani, Lawson Wong

HIERARCHICAL CONTROL SCHEME

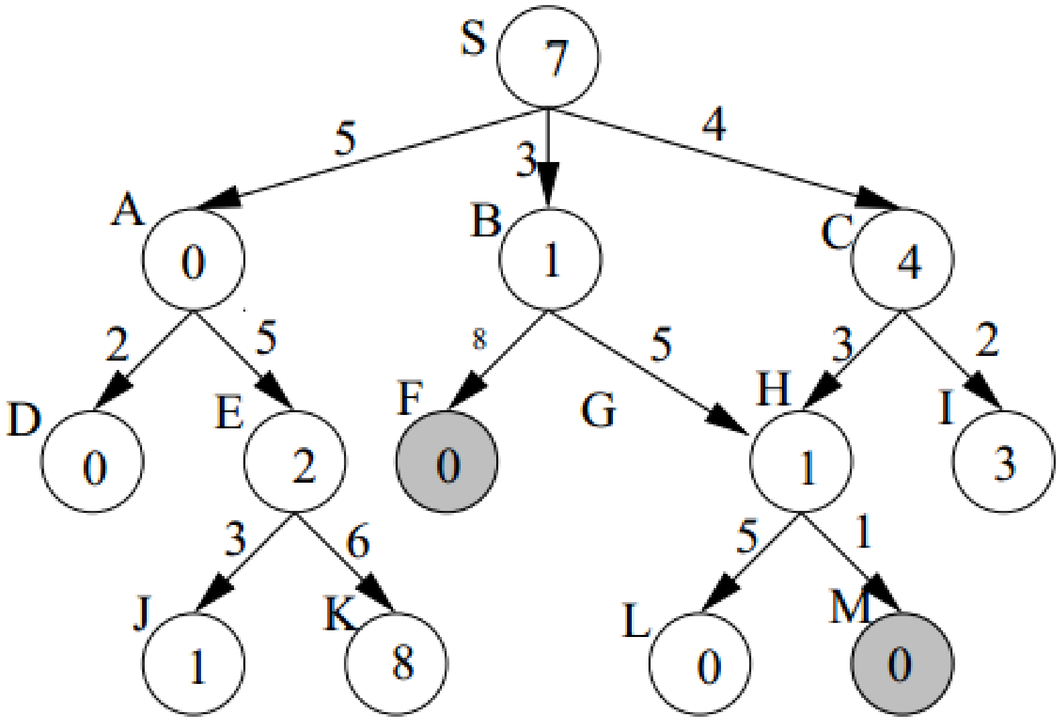
<GAIT GENERATION>

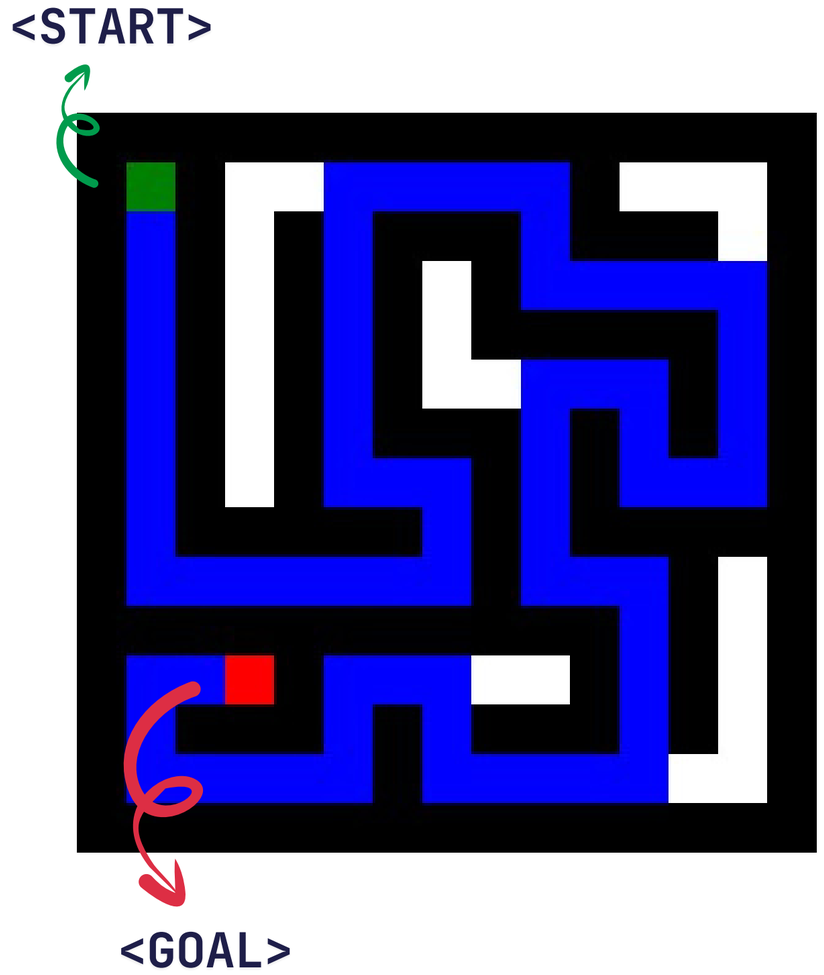
<GLOBAL PLANNING>



<LOCAL PLANNING>

GLOBAL PLANNING: **A* ALGORITHM**





LOCAL PLANNING: **REINFORCEMENT LEARNING**

Reinforcement Learning (RL) is a Machine Learning technique where an **agent** learns to make decisions by interacting with an **environment**, with the goal of maximizing a notion of cumulative **reward** over time.

AGENT → **SAC**

ENVIRONMENT → **CUSTOM SNAKE ENVIRONMENT**

RL: **Soft Actor-Critic (SAC) Algorithm**

ACTOR (The Policy)

A standard MLP (Multi-Layer Perceptron):

Input: Current State

Output: The best Action to take

$$J_\pi(\phi) = \mathbb{E}_{s\sim\mathcal{D}}[\min_{i=1,2} Q_{\theta_i}(s, a_\phi(s)) - \alpha \log \pi_\phi(a_\phi(s)|s)]$$

Entropy

CRITIC (The Evaluator)

Input : State and Action to take

Output : A score (the “Q-value”) estimating the future rewards from the action sampled from the policy of the Actor.

Twin Q-Networks: Two different Critics are trained and the most pessimistic Q-value is chosen.

RL: **Soft Actor-Critic (SAC) Algorithm**

Training Loop

1. Collect Data: The Actor picks an action, the snake moves and *(state, action, reward, next state)* tuple is stored in the replay buffer.

2. Train the Critic: Sample a data from replay buffer and compute its *"target Q-value"*.
Use gradient descent to update the Critic Networks to reduce the MSE between the predicted Q-value and the target Q-value.

3. Train the Actor: Update the Actor using gradient ascent to maximize the Q-values produced by the Critic for its actions.

4. Update Entropy: If the Actor is not exploring enough, increase entropy to encourage more randomness.

5. Repeat steps until convergence.

RL: **CUSTOM SNAKE ENVIRONMENT**

components

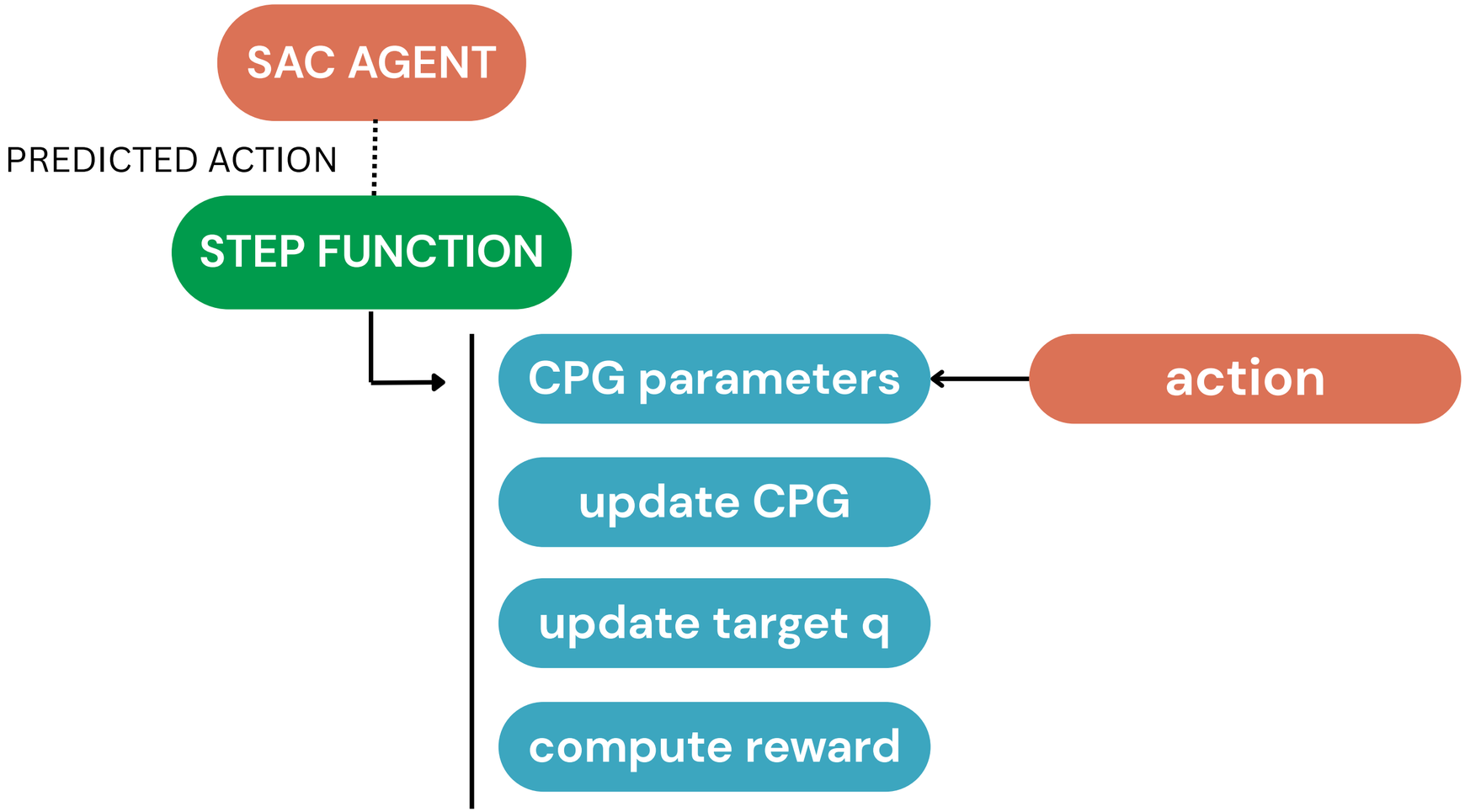
OBSERVATION SPACE

2* n. joints + head position

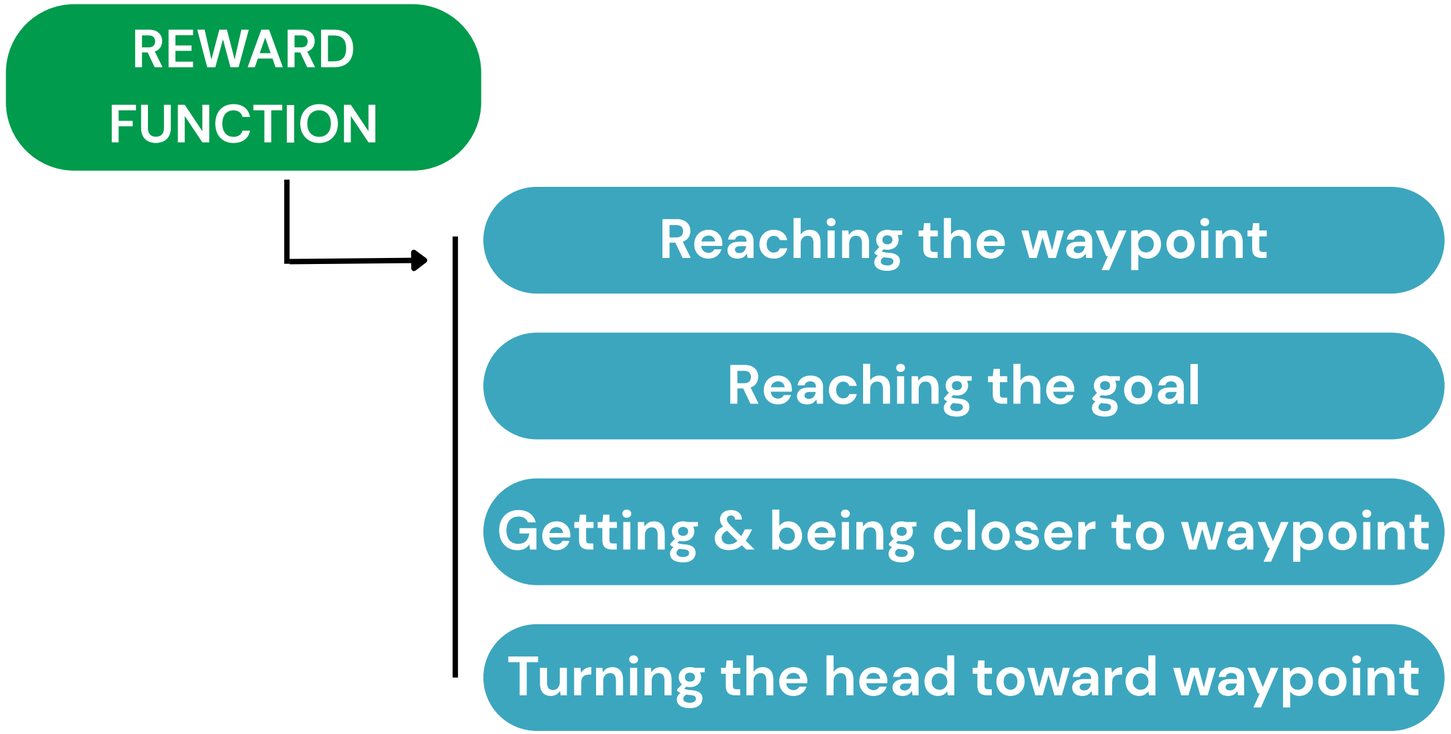
ACTION SPACE

CPG parameters: R, θ, ω, δ

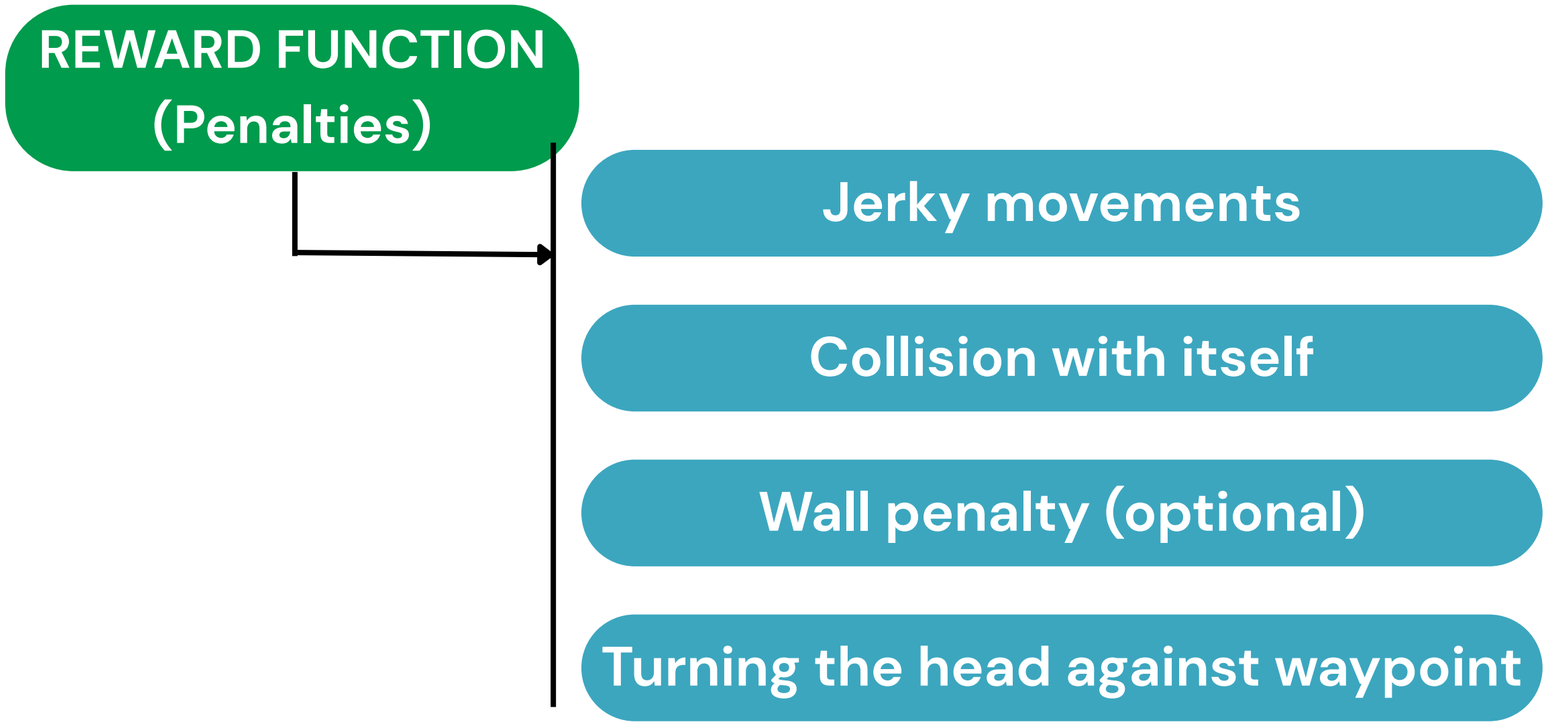
RL: **STEP FUNCTION**



RL: **REWARD FUNCTION**



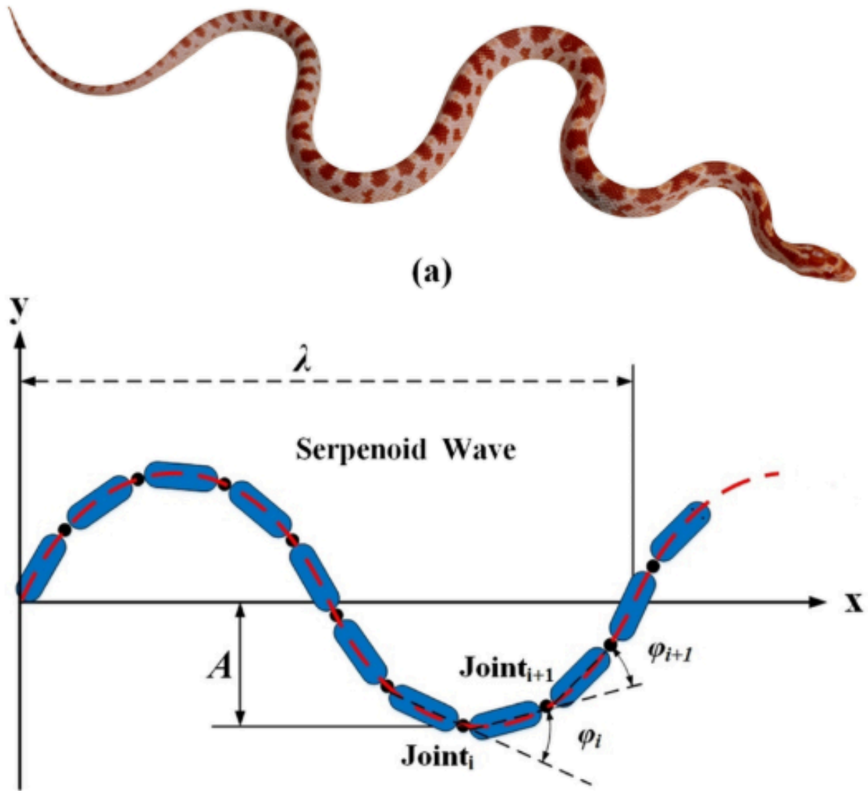
RL: **REWARD FUNCTION**



GAIT GENERATION: **CPG**

Founding Principle

- **CPG is a neural circuit in the vertebrate spinal cord that generates coordinated rhythmic output signals.**
- **Several applications in robot locomotion: multilegged robots, snake robots, ...**
- **Optimization algorithms are often applied to adjust CPG parameters in real-time.**



Wavelike desired joint trajectory

$$x = r \cdot sin\left(\varphi\right) + \delta$$

GAIT GENERATION: **CPG**

Dynamics

Gradient System

$$\frac{\mathrm{d}x}{\mathrm{d}t} = -\frac{\partial V}{\partial x}$$

CPG dynamics

$$\dot{\varphi} = \omega + \mathbf{A} \cdot \varphi + \mathbf{B} \cdot \theta$$

$$\ddot{r} = a \cdot \left[\frac{a}{4} (R - r) - \dot{r} \right]$$

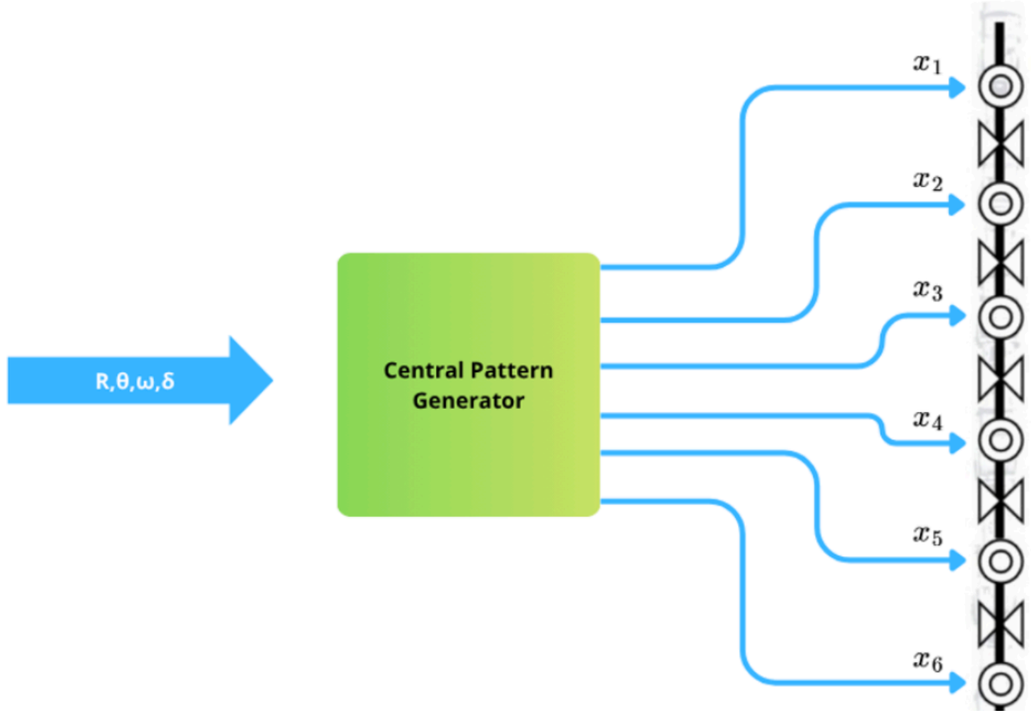
$$\mathbf{A} = \begin{bmatrix} -\mu_1 & \mu_1 & & & \\ \mu_2 & -2\mu_2 & \mu_2 & & \\ & & \ddots & & \\ & & \mu_{n-1} & -2\mu_{n-1} & \mu_{n-1} \\ & & & \mu_n & -\mu_n \end{bmatrix}$$

$$\mathbf{B} = \begin{bmatrix} 1 & & & & \\ -1 & 1 & & & \\ & -1 & \ddots & & \\ & & \ddots & 1 & \\ & & & -1 & 1 \\ & & & & -1 \end{bmatrix}$$

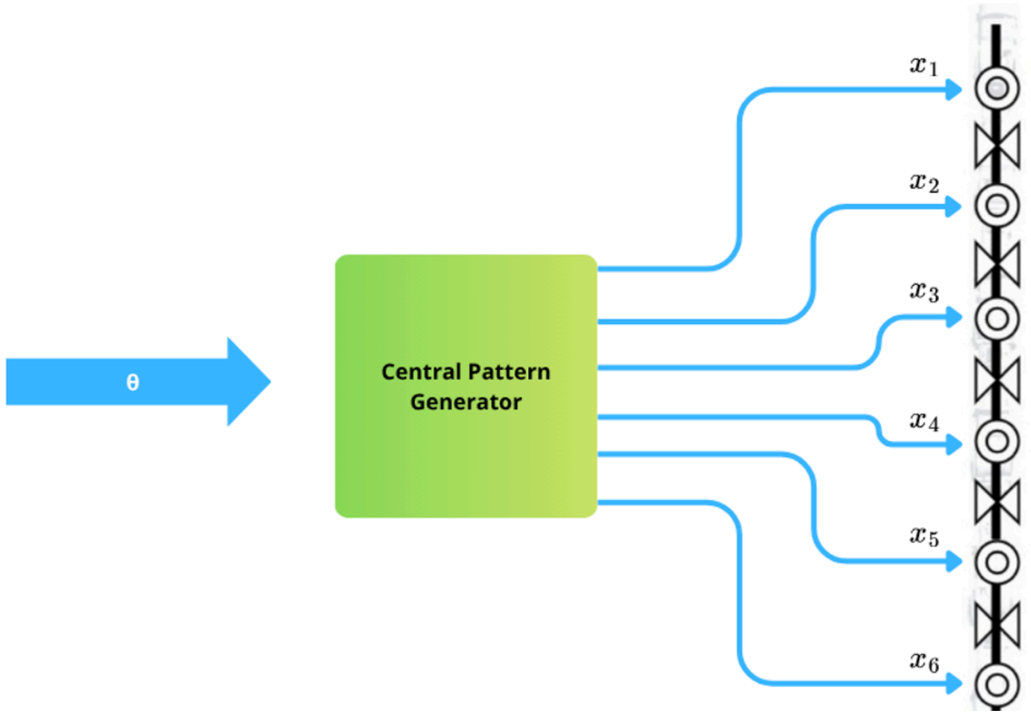
GAIT GENERATION: **CPG**

Learning Parameters

Action Space

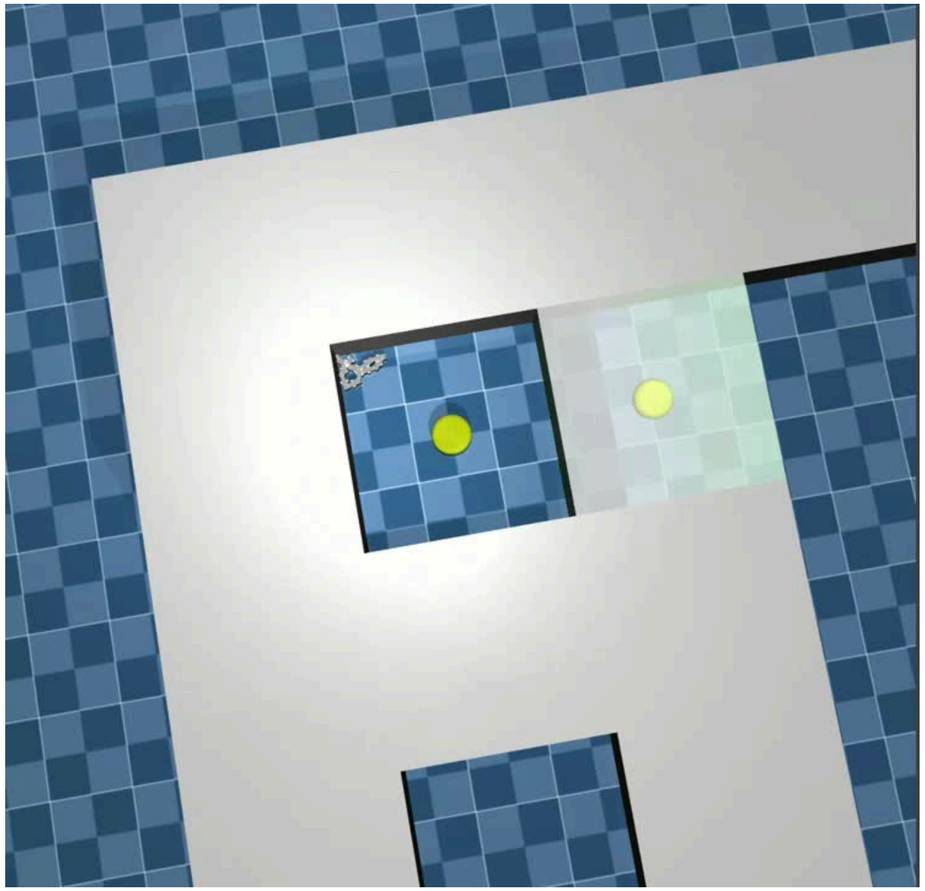


Reduced Action Space



EXPERIMENT 1

- Trained the snake robot to navigate within a maze using Reinforcement Learning (SAC).
- The RL policy controlled all four parameters of the CPG: **R, θ, ω, δ**
- The goal was to learn coordinated movement for navigating complex paths.



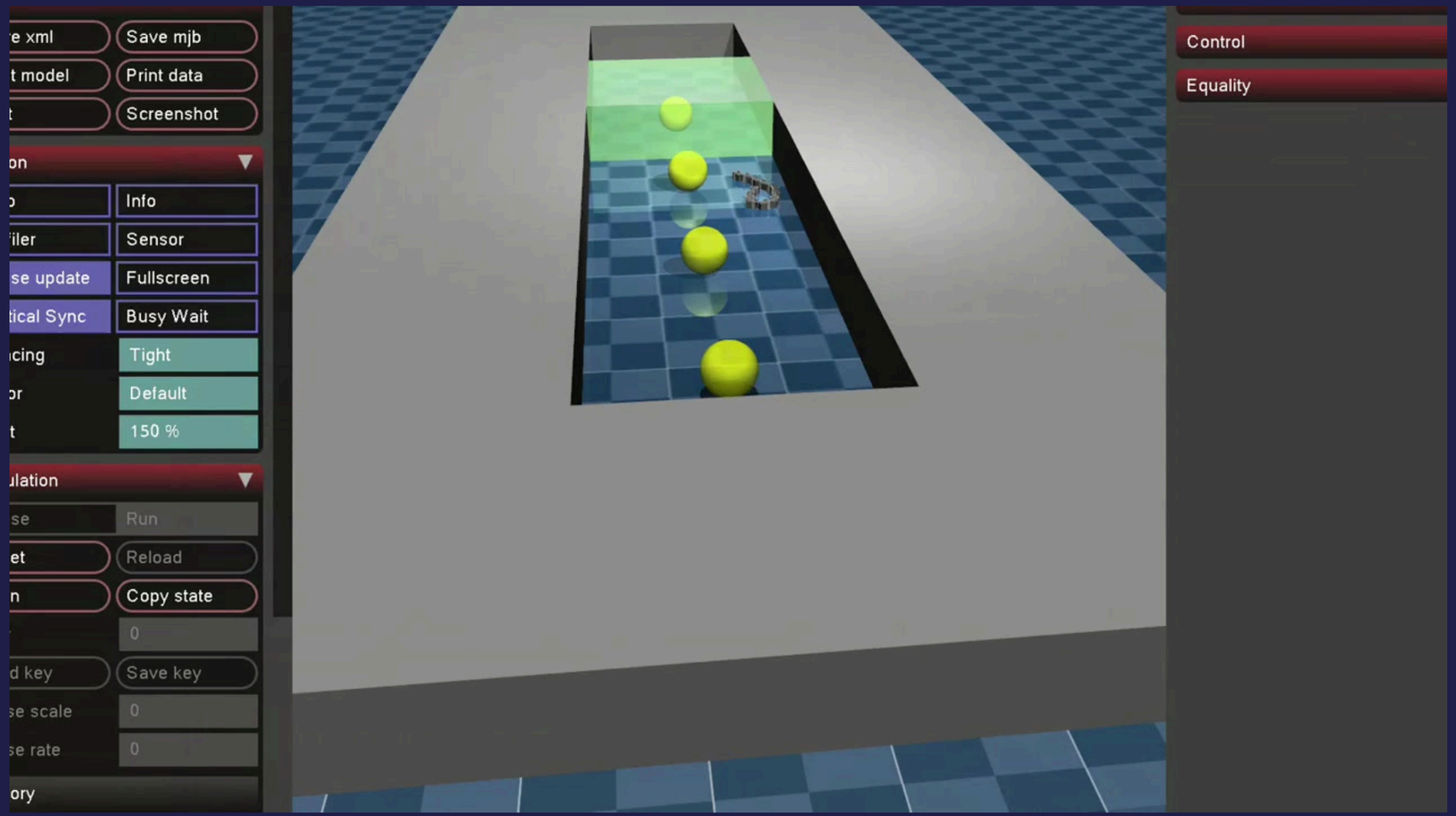
EXPERIMENT 2

- Observed that performance in the maze was poor — the snake showed unstable and inefficient behaviors.
- Hypothesized that the maze was too complex at this stage of learning.
- Simplified the environment to a corridor and retrained the snake with the same 4-parameter control setup.

EXPERIMENT 3

- Despite the simpler environment, the snake still displayed erratic and inefficient movements.
- Decided to **reduce the control complexity** by fixing three CPG parameters: **δ** = 0, **R** = 1, **ω** = 1.
- Only **θ** (phase shift) was learned using Reinforcement Learning.
- This constraint improved learning stability and snake behavior.

RESULT: **RL-CONTROL**



FUTURE WORKS

- Training to enable full maze navigation (First, with the aid of A* waypoints, then without any waypoints).
- Improve simulation stability.
- Real-Life Bot to validate the simulations.
- Improve rewards and penalty.
- Memory and vision modules integration.

References

- Hierarchical RL-Guided Large-scale Navigation of a Snake Robot, 6 Dec 2023
Shuo Jiang , Adarsh Salagame , Alireza Ramezani , Lawson Wong

- Smooth Gait Transition of Body Shape and Locomotion Speed Based on CPG
Control For Snake-like Robot, 4 April 2017, Zhenshan Bing, Long Cheng, Guang Chen, Florian Röhrbein, Kai Huang and Alois Knoll

- Sigmoid transition approach of the central pattern generator-based controller for the snake-like robot, 6 May 2016, Guifang Qiao, Xiulan Wen, Jian Lin, Dongxia Wang and Zhong Wei

- https://mujoco.org/

- https://stable-baselines3.readthedocs.io/en/master/modules/sac.html


Thank you!
PUMA